
Machbarkeitsanalyse und prototypische Entwicklung einer Browsererweiterung zur LLM-basierten Extraktion und Rekonstruktion von UI-Elementen

Bachelorarbeit zur Erlangung des akademischen Grades
Bachelor of Science
im Studiengang Medieninformatik
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln

vorgelegt von: Cosima Hiromi Zink
Matrikel-Nr.: 11158844
Adresse: Steinmüllerallee 1
51643 Gummersbach
cosima_hiromi.zink@smail.th-koeln.de

eingereicht bei: Prof. Christian Noss
Zweitgutachter*in: Prof. Dr. Hoai Viet Nguyen

Gummersbach, 26.02.2026

Kurzfassung

Ziel dieser Arbeit ist die Konzeption und prototypische Entwicklung einer Browsererweiterung zur LLM-basierten Extraktion und Rekonstruktion von UI-Elementen im Kontext der Webentwicklung. Im Rahmen der vorgegebenen neunwöchigen Bearbeitungszeit werden zudem die technischen und konzeptionellen Möglichkeiten sowie Grenzen des entwickelten Prototyps betrachtet. Ein weiterer Schwerpunkt liegt auf der Analyse des Einsatzes von Large Language Models bei der Rekonstruktion bestehender Benutzeroberflächen.

Die entwickelte Browsererweiterung extrahiert strukturierte UI-relevante Informationen aus bestehenden Webseiten und übergibt diese an ein Large Language Model. Zu den erfassten Daten zählen unter anderem DOM-Strukturen, berechnete CSS-Stile und visuelle Informationen in Form von Screenshots sowie Zusatzinformationen, die vom Nutzer bereitgestellt werden. Auf Basis dieser Daten und eines je nach Ausgabtyp gewählten Prompts generiert das LLM eigenständige Webkomponenten oder reduzierte Code-Snippets. Im Vergleich zu bestehenden Werkzeugen, die sich überwiegend auf die Nachbildung visueller Stile beschränken, berücksichtigt der Ansatz dieser Arbeit zusätzlich Aspekte einfacher funktionaler Strukturen.

Die Ergebnisse zeigen, dass die Rekonstruktion statischer und visuell klar definierter UI-Elemente grundsätzlich gut umsetzbar ist und eine hohe visuelle Übereinstimmung mit dem Original aufweist. Gleichzeitig zeigen sich Einschränkungen bei der Rekonstruktion komplexer, zustandsbasierter Interaktionslogiken sowie dynamischer Verhaltensweisen, die über UI-nahe Animationen und responsive Layoutanpassungen hinausgehen. Insgesamt verdeutlicht die Arbeit, dass der entwickelte Ansatz insbesondere als unterstützendes Werkzeug im Webentwicklungsprozess geeignet ist und einen prototypischen Reifegrad erreicht, jedoch keine vollständige Automatisierung produktiver Systeme ermöglicht.

Schlüsselwörter: Browsererweiterung, Large Language Models, Webentwicklung, UI-Rekonstruktion, Prompt Engineering, Prototypische Entwicklung

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
1 Einleitung	1
1.1 Projektkontext	1
1.2 Zielsetzung	2
1.3 Gang der Arbeit	2
2 Theoretischer Hintergrund	3
2.1 Technische Grundlagen von Browsererweiterungen	3
2.2 Large Language Models	4
2.3 Prompt Engineering	7
3 Methodik	8
3.1 Vorgehensweise	8
3.2 Abgrenzungen und Einschränkungen	9
4 Modelle	11
4.1 User Flow	11
4.2 Systemarchitektur der Browsererweiterung	13
5 Iterationen und Bewertung	15
5.1 Technische Umsetzung	15
5.2 Vergleich und Auswahl des Sprachmodells	18
5.3 Konzeptionelle Entscheidungen	21
5.4 Analyse der Rekonstruktionsfähigkeiten	23
6 Fazit und Ausblick	30
6.1 Einordnung der Ergebnisse	30
6.2 Grenzen der Arbeit und Reflexion	31
Literatur	32
Anhang	34
.1 Repository	34

.2	Verwendete Prompt-Vorlagen	34
.2.1	Webkomponente	34
.2.2	Code-Snippet	35
.3	Webseite der Testelemente	35

Abbildungsverzeichnis

4.1	User Flow der Browsererweiterung	12
4.2	Systemarchitektur der entwickelten Browsererweiterung	14
5.1	Das originale UI-Element	19
5.2	Das rekonstruierte UI-Element mittels GPT-5.2	20
5.3	Das rekonstruierte UI-Element mittels Sonnet 4.5	20
5.4	Das rekonstruierte UI-Element mittels Gemini 3 Pro	21
5.5	Leichtes Testelement zur Analyse der Rekonstruktionsfähigkeit	24
5.6	Rekonstruiertes leichtes Testelement zur Analyse der Rekonstruktionsfähigkeit	24
5.7	Mittelschweres Testelement zur Analyse der Rekonstruktionsfähigkeit	25
5.8	Rekonstruiertes mittelschweres Testelement zur Analyse der Rekonstruktionsfähigkeit	26
5.9	Schweres Testelement zur Analyse der Rekonstruktionsfähigkeit	27
5.10	Rekonstruiertes schweres Testelement zur Analyse der Rekonstruktionsfähigkeit	29

1 Einleitung

“Die dynamische Entwicklung und steigende Beliebtheit generativer künstlicher Intelligenz (genKI), besonders durch die Verbreitung und dem Einsatz von ChatGPT, hat das enorme Potenzial dieser Technologie gezeigt, Berufsfelder und Branchen grundlegend transformieren zu können.” (Leible et al., 2024, S. 344). Durch den Einsatz von Sprachmodellen, auch als Large Language Models (LLMs) bezeichnet, wie etwa ChatGPT, ergibt sich zusätzlich die Möglichkeit, den Fokus verstärkt auf die Ideenentwicklung zu legen. Wie Teubner et al. (2023, S. 96) treffend formuliert: “The pace at which innovative ideas have been emerging is breathtaking.” Begünstigt wird dies durch ein sprachliches Ausdrucksvermögen, das in weiten Teilen als menschenähnlich wahrgenommen wird (Teubner et al., 2023).

Besonders die Veröffentlichung von ChatGPT sorgte für eine große Verbreitung von KI-Anwendungsfällen (Teubner et al., 2023). Auch im Bereich der Software- und Webentwicklung lassen sich entsprechende Potenziale erkennen. So ermöglicht die Fähigkeit der Generierung von Programmiercode (Leible et al., 2024) durch Sprachmodelle neue Ansätze zur Unterstützung kreativer und technischer Teilprozesse. Trotz des erkennbaren Potenzials weist generative künstliche Intelligenz Limitationen auf (Leible et al., 2024). Diese gilt es insbesondere bei der Integration solcher Technologien in bestehende Prozesse zu berücksichtigen.

1.1 Projektkontext

Als Bestandteil des Studiengangs Medieninformatik wird diese Bachelorarbeit an der Technischen Hochschule Köln angefertigt. Im Mittelpunkt steht der Einsatz eines Large Language Models zur Extraktion und Rekonstruktion grafischer Benutzeroberflächen im Rahmen einer prototypisch entwickelten Browsererweiterung.

Die Arbeit ist als Machbarkeitsanalyse angelegt. Sie setzt sich mit technischen und konzeptionellen Aspekten auseinander, die für die Entwicklung einer LLM-basierten Browsererweiterung relevant sind. Dabei wird herausgearbeitet, welche Funktionalitäten realisierbar sind und an welchen Stellen bestehende Grenzen erkennbar werden.

Vor diesem Hintergrund ergeben sich die zentralen Forschungsfragen dieser Arbeit:

Welche technischen und konzeptionellen Möglichkeiten und Grenzen bestehen bei der Entwicklung einer LLM-basierten Browserextension im Kontext der Webentwicklung? Wie beeinflussen sie den erreichbaren Entwicklungsstand?

1.2 Zielsetzung

Im Rahmen dieser Arbeit wird ein Prototyp entwickelt, der es ermöglicht, beliebige UI-Elemente auf einer Webseite zu extrahieren und zu rekonstruieren. Dabei übernimmt ein integriertes Large Language Model die Generierung des Codes auf Grundlage strukturierter Eingabedaten. Ziel ist es, die technischen und konzeptionellen Möglichkeiten sowie die bestehenden Grenzen eines solchen Ansatzes herauszuarbeiten. Der entwickelte Prototyp dient dabei nicht nur der praktischen Veranschaulichung der zugrunde liegenden Forschungsfrage, sondern ermöglicht zugleich eine erste Erprobung und Reflexion der Umsetzbarkeit im Anwendungskontext.

1.3 Gang der Arbeit

Die Arbeit gliedert sich zunächst in einen theoretischen Hintergrund, in dem die technischen Grundlagen von Browsererweiterungen, die Funktionsweise und Eigenschaften von Large Language Models sowie zentrale Prinzipien des Prompt Engineerings dargestellt werden. Darauf aufbauend beschreibt das Methodikkapitel die gewählte iterativ-inkrementelle Vorgehensweise sowie die konzeptionellen Abgrenzungen und Rahmenbedingungen der Arbeit. Anschließend werden der zugrunde liegende User Flow sowie die Systemarchitektur der Browsererweiterung erläutert.

Im weiteren Verlauf werden die einzelnen Iterationen des Prototyps, die technische Umsetzung, der Vergleich und die Auswahl des eingesetzten Sprachmodells sowie maßgebliche konzeptionelle Entscheidungen dargestellt. Darauf folgt eine Analyse der Rekonstruktionsfähigkeiten anhand von UI-Elementen unterschiedlicher Komplexität. Den Abschluss der Arbeit bildet das Fazit mit der Einordnung der Ergebnisse sowie einer Reflexion der Grenzen und potentiellen Weiterentwicklungsmöglichkeiten.

2 Theoretischer Hintergrund

Das folgende Kapitel dient der Vermittlung der theoretischen Grundlagen dieser Arbeit und bildet den inhaltlichen Rahmen für die anschließende Analyse und Ergebnisdarstellung. Es führt in grundlegende Konzepte und zentrale Begriffe im Themenfeld von Browsererweiterungen sowie Large Language Models ein und schafft damit ein einheitliches Verständnis der relevanten Grundlagen. Darüber hinaus wird die Relevanz des Themas im Zusammenhang aktueller Entwicklungen und Rahmenbedingungen eingeordnet. Ziel des Kapitels ist es, die theoretischen Voraussetzungen zu schaffen, die für die Einordnung, Bewertung und Diskussion des entwickelten Prototyps erforderlich sind.

2.1 Technische Grundlagen von Browsererweiterungen

Frisbie (2022) bezeichnet Browsererweiterungen metaphorisch als “Parasiten”, da sie auf bestehenden Webseiten operieren, dabei jedoch unabhängig von deren Servern agieren. Diese Eigenständigkeit, kombiniert mit einem hohen Maß an Funktionalität, macht sie zu einem äußerst leistungsfähigen Werkzeug. Sie eröffnen Nutzern zusätzliche Kontrollmöglichkeiten über die individuelle Browsererfahrung, die ohne den Einsatz solcher Erweiterungen nicht realisierbar wären. Beispielsweise verfügen manche Erweiterungen über umfassende Fähigkeiten und können sämtliche Inhalte, die im Browser angezeigt werden, nicht nur einsehen, sondern auch aktiv verändern. Die Verbreitung von Browsererweiterungen lässt sich unter anderem an der Anzahl der im Chrome Web Store verfügbaren Erweiterungen ablesen, die sich auf rund 180.000 beläuft (Frisbie, 2022).

Browsererweiterungen werden innerhalb einer sogenannten sandbox ausgeführt. Ähnlich wie Software-Container läuft der Code dabei in einer isolierten Umgebung, die einerseits den Zugriff auf definierte Browserfunktionen ermöglicht und andererseits die Ausführung von Webtechnologien erlaubt (Mehta, 2016). Erweiterungen rendern Inhalte in einer eigenen Isolationsumgebung, führen JavaScript-Code in einer separaten Laufzeitumgebung aus und greifen auf spezifische Erweiterungs-APIs zu. Die Webseite, auf der die Erweiterung ausgeführt wird, kann diese internen Prozesse nicht wahrnehmen (Frisbie, 2022).

Den grundlegenden Aufbau von Browsererweiterungen beschreibt Frisbie (2022, S. 24) wie folgt: “[...] browser extensions exist as a collection of semi-independent elements.” Das zentrale Element stellt dabei das Manifest dar. Hierbei handelt es sich um eine JSON-Datei, die als Regelwerk der Erweiterung fungiert. Sie enthält unter anderem öffentliche Informationen, Verweise auf Einstiegspunkte, Konfigurationen, Anforderungen, Pattern-Matching-Regeln sowie erweiterungsspezifische Optionen, beispielsweise Tastenkombinationen (Frisbie, 2022). Darüber hinaus definiert das Manifest die benötigten API-Berechtigungen, durch die der Zugriff auf Browserfunktionen reglementiert und potentielle Sicherheitsrisiken, etwa durch Schadsoftware, begrenzt werden sollen (Mehta, 2016).

Ein weiteres zentrales Element, welches Frisbie (2022) beschreibt, sind die Background Scripts, die für die Verarbeitung von Browserereignissen zuständig sind. Hierzu zählen etwa Lebenszyklusereignisse. Die Benutzeroberfläche der Erweiterung wird über die sogenannte Popup Page bereitgestellt. Diese wird beim Öffnen der Erweiterung über der aktuell aufgerufenen Webseite angezeigt und enthält Inhalte, die für den Nutzer leicht erreichbar sein sollen. Bei jedem erneuten Öffnen wird die Seite neu gerendert. Sowohl Background Scripts als auch Popup Pages können auf die Erweiterungs-APIs zugreifen.

Eine weitere wesentliche Komponente stellen die Content Scripts dar. Diese werden im Kontext der jeweiligen Webseite ausgeführt und besitzen Zugriff auf deren DOM-Struktur, wodurch sie Inhalte lesen und verändern können. Zwar ist ihr direkter Zugriff auf bestimmte APIs der Erweiterung eingeschränkt, jedoch können sie über definierte Kommunikationsmechanismen mit anderen Komponenten interagieren. Auf diese Weise wird ein indirekter Zugriff auf die APIs ermöglicht (Frisbie, 2022).

Obwohl Browsererweiterungen bei der Durchführung von Netzwerkanfragen auf denselben technischen Grundlagen wie Webseiten basieren, bestehen hinsichtlich der Origin, der Komponentenstruktur sowie der Authentifizierungsmechanismen spezifische Besonderheiten (Frisbie, 2022). Gleichwohl ermöglichen sie die Anbindung externer serverbasierter Dienste. Large Language Models stellen ein Beispiel für solche Systeme dar, die über entsprechende Programmierschnittstellen angesprochen werden können.

2.2 Large Language Models

Das Large Language Model, im Deutschen als Sprachmodell bezeichnet, wird vom Duden als “Computerprogramm, das natürliche Sprache versteht, verarbeitet und erzeugt” definiert (Dudenredaktion, o.J.). Darüber hinaus zeichnen sich LLMs durch die Fähigkeit aus, ein breites Spektrum an Aufgaben zu bewältigen, was insbesondere auf ihr umfangreiches Vortraining mit sehr großen Textkorpora zurückzuführen ist

(Zhao, 2025). Anders als klassische, aufgabenspezifische Modelle werden sie nicht gezielt für einzelne Anwendungen trainiert, sondern anhand des Ziels optimiert, das jeweils nächste Wort vorherzusagen (Zhao, 2025).

Diese Wortvorhersage erfordert häufig die Lösung unterschiedlicher Teilaufgaben, etwa Stimmungsanalysen, numerische Berechnungen oder Wissensableitungen (Zhao, 2025). Die daraus resultierende vielseitige Problemlösungsfähigkeit hat insbesondere die natürliche Sprachverarbeitung nachhaltig beeinflusst. Traditionelle Aufgaben wie Textzusammenfassungen oder Übersetzungen können inzwischen effektiv durch promptbasiertes Lernen mit LLMs bearbeitet werden und ersetzen zunehmend frühere, spezialisierte Verfahren (Zhao, 2025).

Darüber hinaus sind LLMs in der Lage, komplexe reale Aufgaben zu lösen, die mehrstufige Denkprozesse erfordern, sowie menschliche Anweisungen zu befolgen, die in natürlicher Sprache formuliert sind (Zhao, 2025).

Diese Fähigkeit erstreckt sich auch auf den Bereich der Softwareentwicklung. Da Programmiersprachen formale Sprachen mit klar definierten Regeln darstellen, können LLMs auf Grundlage ihrer generativen Textkompetenz ebenfalls Programmcode analysieren und erzeugen (Schnell, 2024). Aufgrund der strukturierten Syntax und eindeutigen Regelwerke von Programmiersprachen gelingt insbesondere die Analyse bestehenden Codes zuverlässig. Gleichwohl bleibt die fachliche Expertise von Softwareingenieuren weiterhin erforderlich, da LLMs lediglich Teilaspekte der Entwicklungsarbeit übernehmen können (Schnell, 2024). Zwar ist ChatGPT in der Lage, qualitativ hochwertigen Code zu generieren und repetitive oder wenig anspruchsvolle Aufgaben zu unterstützen, jedoch zeigt sich, dass die Analyse vorhandenen Codes in der Regel zuverlässiger funktioniert als die vollständig eigenständige Generierung ohne ausreichenden Kontext (Schnell, 2024).

Schwierigkeiten entstehen insbesondere dann, wenn relevante Informationen, etwa zu Schnittstellen, fehlen. In solchen Fällen kann es zu sogenannten Halluzinationen kommen, bei denen das Modell plausible, jedoch faktisch unzutreffende Inhalte erzeugt (Schnell, 2024). Die höhere Qualität bei der Codeanalyse lässt sich darauf zurückführen, dass dem Modell ein klar abgegrenzter Kontext bereitgestellt wird, während die freie Generierung stärker vom Trainingsmaterial abhängt und somit größeren Unsicherheiten unterliegt. Einschränkungen zeigen sich insbesondere bei selten verwendeten Programmiersprachen oder spezialisierten Frameworks (Schnell, 2024).

Zur Reduktion solcher Problematiken empfiehlt es sich, dem Modell zu Beginn eine spezifische Rolle oder einen klar definierten Kontext zuzuweisen sowie präzise und lösbare Anforderungen zu formulieren. Fehlt eine solche Kontextualisierung, kann das generierte Ergebnis deutlich von den intendierten Anforderungen abweichen (Schnell, 2024).

Angesichts dieser Leistungsfähigkeiten und der zugleich bestehenden Unsicherheiten im praktischen Einsatz rückte mit der raschen Verbreitung von ChatGPT zunehmend die Frage in den Fokus, in welchem Umfang KI-Systeme künftig berufliche Tätigkeiten übernehmen oder verändern könnten (Schnell, 2024).

In diesem Zusammenhang sind neben den aufgezeigten Potentialen auch die damit verbundenen Risiken zu berücksichtigen, insbesondere im Hinblick auf ihren Einfluss auf die Arbeitswelt. Zu den zentralen Problembereichen zählen Intransparenz, Qualitätsunsicherheiten sowie eine potentielle Abhängigkeit von Anbietern (Barenkamp, 2023). Unter Intransparenz ist dabei die mangelnde Nachvollziehbarkeit der Entscheidungs- und Generierungsprozesse zu verstehen, da Sprachmodelle in der Regel weder ihre verwendeten Quellen offenlegen noch Aussagen zur Wahrscheinlichkeit oder Verlässlichkeit ihrer Ergebnisse bereitstellen (Barenkamp, 2023). Darüber hinaus besteht die Gefahr einer unreflektierten Nutzung durch Mitarbeitende, was zu einem schleichenden Kompetenzverlust führen kann (Barenkamp, 2023).

Vor dem Hintergrund dieser Chancen und Risiken untersuchen aktuelle Studien die konkrete Leistungsfähigkeit von LLM-basierten Chatbots in spezifischen Anwendungskontexten. So vergleichen Smutny und Bojko (2024) verschiedene LLM-Chatbots hinsichtlich ihrer Eignung für Webentwicklungsaufgaben. In der Untersuchung wurden unter anderem die Chatbots Copilot, GPT-3.5 und Gemini analysiert. Der Vergleich fokussierte sich auf die Codegenerierung, wobei die Modelle die Aufgabe erhielten, eine einfache HTML-Webseite mit einem PHP-Formular unter Einbindung einer MySQL-Datenbank zu erstellen. Bewertet wurden dabei Kriterien wie Genauigkeit, Benutzerfreundlichkeit, Kreativität, korrekte Datenbankintegration sowie das Kontextverständnis.

Die Ergebnisse zeigen, dass die untersuchten Chatbots in der Lage sind, komplexe und mehrstufige Aufgaben erfolgreich zu bearbeiten. Gleichzeitig wird deutlich, dass weiterhin Optimierungspotenzial besteht. Insgesamt verdeutlichen die Resultate jedoch, dass LLM-Chatbots ein relevantes Unterstützungspotential für Webentwickler besitzen und zur Steigerung der Produktivität beitragen können. Darüber hinaus differenziert die Studie zwischen dialogorientierten Chatbots und Programmierassistenten. Während dialogorientierte Modelle wie ChatGPT oder Copilot ein ausgeprägtes Verständnis von Eingabeaufforderungen aufweisen, benutzerfreundliche Inhalte generieren und ansprechende Designs entwickeln können, orientieren sich Programmierassistenten stärker an syntaktischen Anforderungen, wodurch sie strukturelle Vorgaben besonders präzise einhalten (Smutny & Bojko, 2024).

2.3 Prompt Engineering

Die Leistungsfähigkeit der generierten Ergebnisse hängt maßgeblich von der Ausgestaltung der formulierten Eingabeaufforderung ab (Gupta et al., 2025). Eine solche Eingabeaufforderung, auch als Prompt bezeichnet, stellt die dem Modell übermittelte Anweisung oder Anfrage dar, auf deren Grundlage das LLM eine passende Ausgabe generiert (Gupta et al., 2025). Prompts übernehmen somit eine zentrale Steuerungsfunktion in der Interaktion zwischen Mensch und Sprachmodell.

Bei der Konzeption eines Prompts sind verschiedene Gestaltungsaspekte zu berücksichtigen, die sowohl das Design als auch die strukturelle Ausformulierung betreffen. Ein Prompt dient dabei nicht ausschließlich als Instruktion, sondern stellt zugleich den kontextuellen Rahmen für die Generierung der Ausgabe bereit (Gupta et al., 2025). Als grundlegendes Prinzip benennen Gupta et al. (2025) die Klarheit und Spezifität der Eingabe. Damit die Antwort den Erwartungen der Nutzenden entspricht, ist eine präzise und klar strukturierte Formulierung erforderlich. Fehlt diese, besteht Gefahr, dass das Sprachmodell unzutreffende oder irrelevante Ergebnisse erzeugt (Gupta et al., 2025).

Der Kontext liefert dem LLM zusätzliche Informationen zur Ausgabegenerierung, beispielsweise in Form vorangehender Konversationen oder domänenspezifischen Wissens. Insbesondere bei fachlich anspruchsvollen Aufgaben ist eine ausreichende Kontextualisierung entscheidend, um angemessene und qualitativ hochwertige Ergebnisse zu erzielen. Entsprechend sollten die im Prompt enthaltenen Angaben so konkret und detailliert wie möglich formuliert werden (Gupta et al., 2025).

Darüber hinaus sollten Prompts aufgabenspezifisch gestaltet sein, da sich je nach Aufgabenart auch die geeignete Struktur der Eingabe unterscheidet. Die Formulierung sollte daher die jeweilige Aufgabenstellung widerspiegeln (Gupta et al., 2025). Neben inhaltlichen Kriterien ist zudem die Länge der Eingabeaufforderung zu berücksichtigen. Eine zu geringe Informationsmenge kann zu unvollständigen Ausgaben führen, während übermäßig umfangreiche Prompts das Modell potenziell überfordern und die Ergebnisqualität beeinträchtigen können (Gupta et al., 2025).

3 Methodik

Zur wissenschaftlichen Fundierung der vorliegenden Arbeit sowie zur Beantwortung der formulierten Forschungsfragen wurden spezifische methodische Ansätze herangezogen. Diese dienten sowohl der systematischen Strukturierung des konzeptionellen Vorgehens als auch der prototypischen Umsetzung im Rahmen einer Machbarkeitsanalyse der entwickelten Browsererweiterung. Die gewählten Methoden ermöglichen eine nachvollziehbare und transparente Darstellung des Entwicklungsprozesses und bilden die Grundlage für die anschließende Analyse und Bewertung der Ergebnisse.

3.1 Vorgehensweise

Im Rahmen der vorliegenden Machbarkeitsanalyse wird die Entwicklung des Prototyps nach einem agilen Prozessmodell gestaltet. Im Mittelpunkt steht dabei die schrittweise Realisierung und Validierung zentraler Funktionalitäten, während die Dokumentation den Entwicklungsprozess begleitet (Hanser, 2010). Die Wahl eines agilen Vorgehensmodells begründet sich insbesondere durch dessen Flexibilität und Anpassungsfähigkeit gegenüber neuen Erkenntnissen im Entwicklungsverlauf. Der iterativ-inkrementelle Charakter ermöglicht die Gliederung der Entwicklung in zeitlich begrenzte Zyklen, sogenannte Iterationen, in denen neue Funktionalitäten implementiert und überprüft werden. Auf diese Weise wird eine kontinuierliche Weiterentwicklung und Optimierung der Lösung unterstützt (Hanser, 2010).

Zur Organisation der einzelnen Arbeitsschritte wurde ein Kanban-Board eingesetzt. Diese Methode dient der strukturierten Planung und kontinuierlichen Steuerung von Aufgaben und wird seit 2007 im Kontext schlanker Softwareentwicklung angewendet (Epping, 2011). Durch die Visualisierung des Arbeitsfortschritts ermöglicht das Kanban-Board eine transparente Priorisierung sowie eine gleichmäßige Verteilung der anstehenden Tätigkeiten.

Die Projektstruktur orientiert sich an vier definierten Meilensteinen, die jeweils zentrale Zielzustände im Entwicklungsprozess markieren und als zeitliche Referenzpunkte dienen. Der erste Meilenstein dient der Erarbeitung der wissenschaftlichen Grundlagen sowie der konzeptionellen Ausgestaltung des Vorhabens. Mit dem zweiten Meilenstein wird der erfolgreiche Beginn der prototypischen Umsetzung erreicht, bei dem zentrale

Funktionalitäten in Form einzelner Proof of Concepts (POCs) entwickelt und technisch validiert werden. Der dritte Meilenstein umfasst die Integration der zuvor entwickelten POCs zu einem Prototyp sowie die begleitende wissenschaftliche Dokumentation und Aufbereitung der Ergebnisse. Der abschließende vierte Meilenstein dient der Finalisierung der Arbeit, einschließlich der inhaltlichen Überarbeitung und formalen Anpassungen.

Innerhalb der prototypischen Entwicklungsphase werden vier Kernfunktionalitäten definiert, die in ihrer Gesamtheit die strukturelle Grundlage des Prototyps bilden. Die Implementierung erfolgt aufeinander aufbauend, sodass die Reihenfolge der Umsetzung aus technischer und konzeptioneller Sicht essenziell ist.

Der erste POC adressiert die Auswahl von UI-Elementen über eine DOM-basierte Interaktion. Hierfür wird ein sogenannter Select Mode implementiert, der es Nutzern ermöglicht, ein beliebiges Element einer Webseite gezielt auszuwählen. Darauf aufbauend fokussiert der zweite Proof of Concept die strukturierte Übergabe relevanter HTML, CSS sowie kontextueller Informationen an das LLM und damit die grundlegende Kommunikation mit dem Sprachmodell.

Im dritten POC wird die Möglichkeit geschaffen, zusätzlich Informationen zum JavaScript-Verhalten des ausgewählten UI-Elements zu übermitteln, sodass auch dynamische Aspekte der Benutzeroberfläche bei der Generierung berücksichtigt werden können. Der vierte und abschließende POC erweitert dieses Vorgehen um die Option, dem Prompt einen Screenshot des Elements beizufügen, um visuelle Merkmale in die Rekonstruktion einfließen zu lassen.

3.2 Abgrenzungen und Einschränkungen

Der zeitlich auf neun Wochen begrenzte Bearbeitungszeitraum erfordert eine frühzeitige Festlegung inhaltlicher Schwerpunkte sowie eine klare Abgrenzung des Projektumfangs. Im Mittelpunkt der Arbeit steht die Machbarkeitsanalyse der LLM basierten Extraktion und Rekonstruktion von UI-Elementen, nicht jedoch die Entwicklung einer marktreifen Anwendung. Entsprechend erfolgt eine prototypische Umsetzung der Browsererweiterung mit den zuvor definierten Kernfunktionalitäten. Eine langfristige iterative Optimierung oder umfassende funktionale Erweiterung ist im Rahmen der vorliegenden Arbeit nicht vorgesehen.

Zur Sicherstellung eines ausreichenden Entwicklungsfokus wird auf eine formale Evaluation verzichtet. Stattdessen werden interne Funktionstests durchgeführt, die die grundsätzliche Funktionsfähigkeit des Prototyps in ausgewählten Anwendungsszenarien

nachvollziehbar machen. Eine systematische Abdeckung unterschiedlichster Webseitenstrukturen oder Nutzungskontexte erfolgt jedoch nicht, sodass die Aussagekraft auf die getesteten Szenarien beschränkt bleibt.

Aufgrund des begrenzten Projektumfangs wird ein vortrainiertes LLM über eine externe Programmierschnittstelle eingesetzt. Eine eigenständige Modellentwicklung oder eine trainingsbasierte Modellanpassung ist nicht Bestandteil der Arbeit. Hieraus ergibt sich eine technische Abhängigkeit vom jeweiligen Dienstanbieter.

4 Modelle

Im folgenden Kapitel werden zwei Modelle vorgestellt, die einen ersten strukturierten Einblick in die konzipierte und entwickelte Browsererweiterung ermöglichen. Zunächst wird der User Flow beschrieben, der die Interaktionsabfolge aus Sicht der Nutzenden darstellt und aufzeigt wie diese durch die Anwendung navigieren sowie welche Handlungsoptionen ihnen zur Verfügung stehen. Anschließend wird die grundlegende Architektur der Erweiterung dargestellt. Dabei werden die zentralen Systemkomponenten sowie deren Zusammenwirken in einer zusammenfassenden Abbildung visualisiert.

4.1 User Flow

Der User Flow (vgl. Abbildung 4.1) stellt ein Modell dar, das die aufeinanderfolgenden Handlungsschritte sowie die möglichen Entscheidungsoptionen innerhalb der Browsererweiterung strukturiert abbildet. Er dient der systematischen Darstellung, dem besseren Verständnis sowie der Analyse der Interaktionsabläufe aus Perspektive der Nutzenden.

Die dargestellte Interaktionssequenz beginnt mit der Aktivierung des Select-Modus. Dadurch wird es ermöglicht, im nächsten Schritt ein beliebiges UI-Element auf einer frei gewählten Webseite auszuwählen. Nach erfolgreicher Selektion kann eine Anfragekonfiguration bestimmt werden, die als Grundlage für die nachfolgende Verarbeitung durch das LLM dient und die Ausgestaltung der generierten Ausgabe maßgeblich beeinflusst.

Anschließend folgen zwei optionale Entscheidungsschritte. Zunächst besteht die Möglichkeit, zusätzliche textuelle Hinweise oder Kontextinformationen zu übermitteln, um die Generierung weiter zu spezifizieren. Darüber hinaus kann entschieden werden, ob ergänzend ein Screenshot an das Sprachmodell übertragen werden soll. In diesem Fall ist der Upload einer Bilddatei im PNG- oder JPEG-Format vorgesehen.

Nachdem diese Konfigurationsschritte abgeschlossen sind, kann die Generierung des Quellcodes ausgelöst werden. Das LLM rekonstruiert daraufhin das ausgewählte UI-Element. Abschließend besteht die Möglichkeit, den erzeugten Code zu kopieren und damit für weitere Verwendungen zu übernehmen.

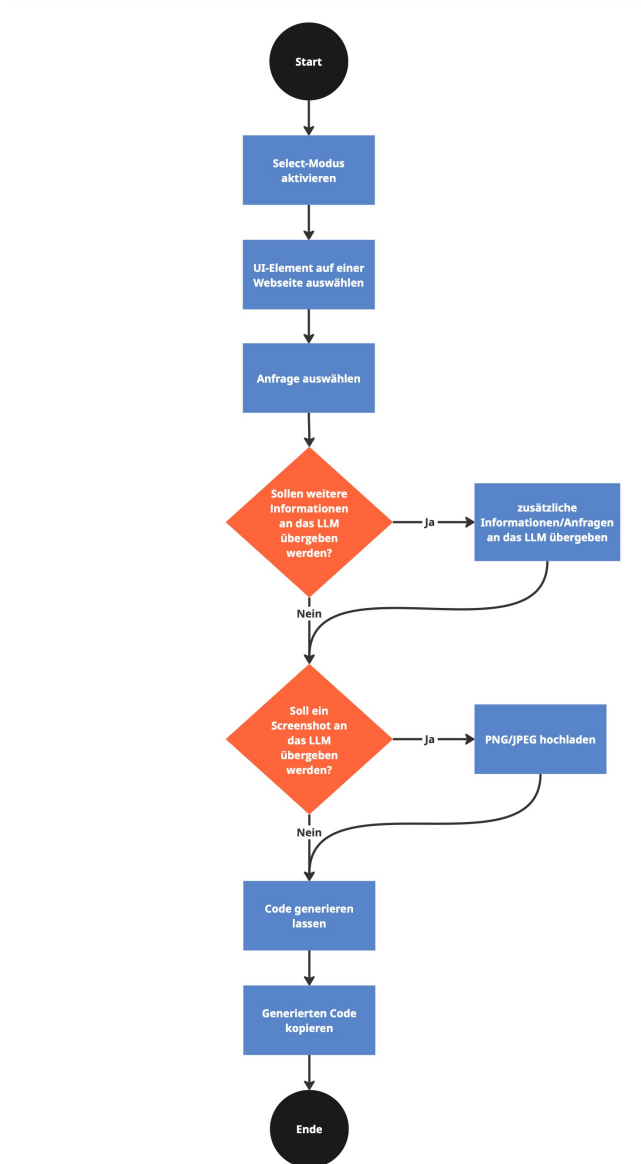


Abbildung 4.1: User Flow der Browsererweiterung

4.2 Systemarchitektur der Browsererweiterung

Die folgende Abbildung (vgl. Abbildung 4.2) stellt die Systemarchitektur der im Rahmen dieser Arbeit entwickelten Browsererweiterung dar. Sie visualisiert die zentralen Systemkomponenten, deren funktionale Beziehungen sowie die wesentlichen Datenflüsse zwischen den einzelnen Einheiten. Darüber hinaus werden die logischen Systemgrenzen zwischen Webbrowser, Browsererweiterung und externen Diensten verdeutlicht.

Das Architekturmodell gliedert sich somit in drei übergeordnete Bereiche. Der erste Bereich umfasst den Webbrowser, konkret Google Chrome, für den die Erweiterung implementiert wurde. Innerhalb des Browsers werden Webseiten dargestellt, auf denen Nutzer gezielt UI-Elemente auswählen können, die rekonstruiert werden sollen. Die Interaktion des Nutzers mit der Webseite stellt dabei den Ausgangspunkt des Verarbeitungsprozesses dar und initiiert die anschließende Datenextraktion durch die Erweiterung.

Die zweite zentrale Komponente stellt die Browsererweiterung selbst dar. Sie besteht aus dem Popup-User-Interface, dem Content Script sowie der Schnittstelle zum lokalen Speicher über `chrome.storage.local`. Das Content Script wird im Kontext der jeweils geöffneten Webseite ausgeführt und ist für die DOM-Selektion sowie die Extraktion relevanter Informationen verantwortlich. Hierbei greift es direkt auf das DOM der Webseite zu, erfasst das ausgewählte UI-Element und überführt die gewonnenen Informationen in eine strukturierte Datenrepräsentation. Es erstellt auf Basis des ausgewählten Elements eine strukturierte Datenrepräsentation (Selektions-Payload), die unter anderem strukturelle, stilistische und positionsbezogene Informationen enthält. Diese Daten bilden die Grundlage für die anschließende LLM-gestützte Rekonstruktion und werden im lokalen Speicher abgelegt.

Der lokale Speicher übernimmt dabei die Funktion einer persistenten Kommunikationsschnittstelle innerhalb der Erweiterung, indem er den Austausch zwischen Content Script und Popup-Komponente ermöglicht. Neben dem Selektions-Payload werden dort auch der generierte Code sowie optional hochgeladene Screenshot-Daten gespeichert.

Das Popup-User-Interface dient als Interaktions- und Steuerungskomponente der Browsererweiterung. Es koordiniert die Aktivierung und Deaktivierung des Auswahlmodus in Abstimmung mit dem Content Script, wodurch die Interaktion zwischen Benutzeroberfläche und Webseitenkontext gesteuert wird. Zudem interagiert der Nutzer über die Oberfläche direkt mit dem System, indem er den Prompt konfiguriert, optional einen Screenshot des ausgewählten Elements hochlädt sowie die Code-Generierung auslöst. Zu diesem Zweck werden die zuvor im lokalen Speicher abgelegten Elementdaten ausgelesen und gemeinsam mit der Prompt-Konfiguration zu einer LLM-Anfrage

zusammengeführt. Der vom Sprachmodell erzeugte Code wird anschließend im Popup angezeigt und ebenfalls persistent im lokalen Speicher abgelegt.

Die Kommunikation mit dem externen Dienst erfolgt über einen zwischengeschalteten Proxy-Server. Dieser nimmt die vom Popup gesendete Anfrage entgegen und leitet sie an die LLM-Schnittstelle weiter. Die LLM-API verarbeitet die übermittelten strukturierten Elementdaten sowie gegebenenfalls zusätzliche Bildinformationen und generiert auf dieser Grundlage den entsprechenden Code. Die erzeugte Antwort wird über den Proxy-Server an die Browsererweiterung zurückgegeben.

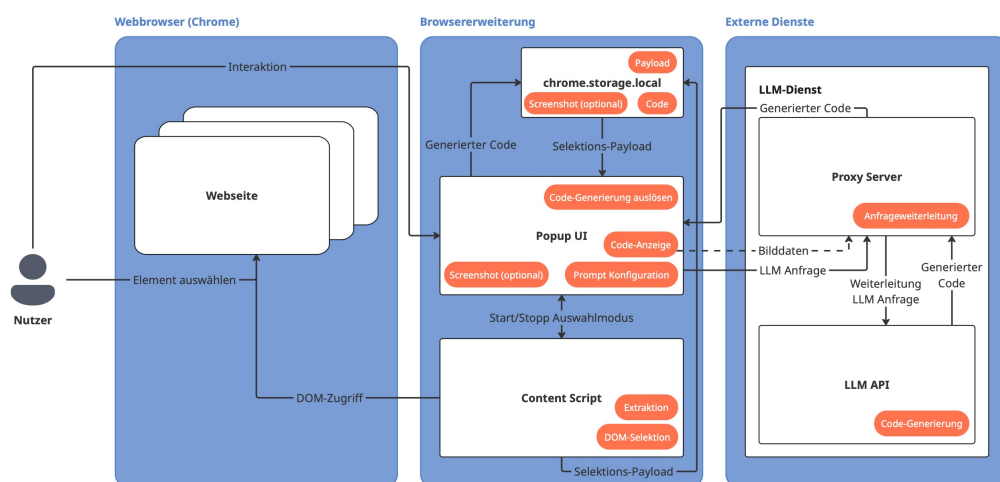


Abbildung 4.2: Systemarchitektur der entwickelten Browsererweiterung

5 Iterationen und Bewertung

Die Entwicklung der Browsererweiterung erfolgt in mehreren aufeinander aufbauenden Iterationen. Ziel dieses Vorgehens ist die Analyse der Machbarkeit einer LLM-basierten Extraktion und Rekonstruktion von UI-Elementen im Kontext der Webentwicklung.

In diesem Kapitel werden die technische Umsetzung des Prototyps sowie die zugrunde liegenden konzeptionellen Entscheidungen dargestellt. Zudem wird die Auswahl des eingesetzten Sprachmodells begründet und im Hinblick auf seine Eignung bewertet. Auf dieser Basis werden die zentralen Möglichkeiten und Grenzen der entwickelten Lösung eingeordnet.

5.1 Technische Umsetzung

Aufgrund der iterativ-inkrementellen Vorgehensweise erfolgt die Entwicklung in klar abgegrenzten Schritten. Diese Iterationen werden in Form von Proof of Concepts umgesetzt, die jeweils zentrale Funktionalitäten des Prototyps abbilden und schrittweise erweitern. Gleichzeitig dienen sie der systematischen Identifikation technischer Möglichkeiten und Grenzen der entwickelten Lösung.

Zur Schaffung einer technischen Grundlage wird im ersten POC ein Selektionsmechanismus implementiert. Dieser ermöglicht die Auswahl eines beliebigen DOM-Elements auf einer Webseite und bildet damit die Kernfunktionalität der Erweiterung. Im Zuge der Selektion werden strukturelle sowie visuelle Eigenschaften des gewählten Elements extrahiert und in einer strukturierten JSON-Payload gespeichert. Diese Payload stellt die Basis für die nachfolgende Verarbeitung durch ein Large Language Model dar.

Der Selektionsmodus kann durch die Nutzenden über das Popup der Browsererweiterung aktiviert beziehungsweise deaktiviert werden. Während der Aktivierung wird beim Überfahren eines Elements eine visuelle Markierung angezeigt, die nach der Auswahl dauerhaft bestehen bleibt, bis der Modus beendet wird. Um unbeabsichtigte Seiteninteraktionen während der Auswahl zu verhindern, werden, solange der Selektionsmodus aktiv ist, Klick-Events von der Erweiterung abgefangen und die Standardausführung unterdrückt. Dadurch wird sichergestellt, dass keine Navigationen oder andere skriptgesteuerte Aktionen der Webseite ausgelöst werden.

Die erzeugte Payload enthält eine gezielte Auswahl relevanter Informationen. Eine vollständige Rekonstruktion der gesamten DOM-Struktur wird bewusst vermieden, da dies zu erheblichen Datenmengen führt und die anschließende Verarbeitung durch ein LLM erschweren würde. Stattdessen erfolgt eine kuratierte Extraktion solcher Inhalte, die für die spätere Rekonstruktion des UI-Elements wesentlich sind. Hierzu zählen unter anderem die HTML-Struktur, ausgewählte Attribute, Textinhalte, berechnete Stilinformationen sowie positionsbezogene Angaben.

Zur Berücksichtigung des aktuellen visuellen Zustands werden berechnete CSS-Eigenschaften mittels `getComputedStyle` erfasst. Ergänzend wird ein CSS-basierter Selektor generiert, der die Position des Elements innerhalb der DOM-Hierarchie beschreibt. Dieser Selektor dient jedoch lediglich als referenzieller Hinweis, da sich Dokumentationsstrukturen und Klassennamen dynamisch verändern können.

Im Hinblick auf die Grenzen der Extraktion ist insbesondere hervorzuheben, dass kein Zugriff auf zugrunde liegende JavaScript-Logik besteht. Dynamische Interaktionen, Ereignisbehandlungen oder zustandsabhängige Funktionalitäten können daher nur eingeschränkt berücksichtigt werden. Zusätzlich unterliegt die Erweiterung sicherheits- und plattformbedingten Restriktionen des Browser-Ökosystems, wodurch bestimmte Inhalte oder Kontexte nicht zugänglich sind. Bei komplex verschachtelten Layoutstrukturen kann zudem ein partieller Kontextverlust auftreten, da ausschließlich das selektierte Element sowie ein begrenzter struktureller Rahmen erfasst werden.

Trotz dieser Einschränkungen bietet der Ansatz wesentliche Vorteile. Die strukturierte Extraktion zentraler UI-Informationen erfolgt zuverlässig und basiert auf standardisierten Web-APIs auf DOM-Ebene. Sie ist somit grundsätzlich nicht an ein bestimmtes Frontend-Framework gebunden. Dennoch bestehen plattform- und sicherheitsbedingte Einschränkungen, etwa bei geschlossenen Shadow-DOM-Strukturen oder Cross-Origin-Kontexten. Insgesamt bildet der Selektionsmechanismus eine tragfähige Grundlage für die weiteren Iterationen des Prototyps.

Im zweiten Proof of Concept wird die Anwendung um die Anbindung eines Large Language Models zur Code-Generierung erweitert. Ziel dieser Iteration ist die Rekonstruktion des zuvor ausgewählten UI-Elements auf Basis der extrahierten Struktur- und Stilinformationen. Durch die Integration entsteht eine durchgängige Verarbeitungskette. Nach der Auswahl eines Elements wird eine strukturierte Payload erzeugt, die als Eingabe für das LLM dient. Anschließend wählt der Nutzer eine vordefinierte Aufgabenbeschreibung aus und initiiert die Anfrage. Nach erfolgter Modellverarbeitung wird der generierte Code in der Benutzeroberfläche angezeigt und kann zur Weiterverwendung kopiert werden.

Zur Gewährleistung einer sicheren und kontrollierten Kommunikation mit der LLM-API wird ein separater Proxy-Server eingesetzt. Dieser kapselt die API-Anfrage, schützt

den API-Schlüssel vor Offenlegung im Client und übernimmt zentrale Aufgaben wie Fehlerbehandlung und Anfrageverwaltung. Auf diese Weise wird eine klare Trennung zwischen Frontend und serverseitiger Modellanbindung geschaffen.

Zur Erhöhung der Robustheit wird zudem ein serverseitiger Wiederholungsmechanismus implementiert. Tritt bei der Anfrage an das Sprachmodell ein temporärer Fehler auf, wird die Anfrage automatisch erneut ausgeführt. Dieses Verfahren ist insbesondere bei instabilen Netzwerkverbindungen oder kurzfristigen API-Problemen vorteilhaft und trägt zur Stabilität des Prototyps bei.

Trotz der erfolgreichen End-to-End-Integration bestehen weiterhin grundlegende technische Grenzen. Auch wenn der generierte Code syntaktisch korrekt ist, kann die visuelle Darstellung vom ursprünglichen UI-Element abweichen. Dies ist insbesondere auf Interpretationsspielräume des Modells sowie auf die zuvor beschriebenen Einschränkungen bei der Extraktion zurückzuführen. In nachfolgenden Iterationen werden daher zusätzliche Maßnahmen implementiert, die eine höhere visuelle Annäherung an das Originalelement unterstützen sollen.

In einer weiteren Iteration wird ein zusätzlicher Eingabekanal in Form eines Freitextfeldes implementiert, der eine differenzierte Beschreibung visueller Besonderheiten und möglicher Zustandsänderungen des ausgewählten UI-Elements ermöglicht. Diese Zusatzinformationen werden gemeinsam mit der strukturierten Payload an das Sprachmodell übermittelt und bei der Code-Generierung berücksichtigt. Sowohl der Zusatzhinweis als auch ein bereits generierter Output werden beim erneuten Aktivieren des Selektionsmodus zurückgesetzt, um eine Vermischung unterschiedlicher Kontexte zu vermeiden.

Obgleich diese Funktion die Flexibilität der Anwendung erhöht und eine feinere visuelle Steuerung ermöglicht, bestehen weiterhin Grenzen. Nutzerhinweise können unpräzise oder widersprüchlich formuliert sein und führen daher nicht zwangsläufig zu einer konsistenteren Rekonstruktion. Die Qualität der Generierung bleibt somit in gewissem Maße von der Spezifität der Eingaben abhängig.

Zur weiteren Steigerung der visuellen Ähnlichkeit wird im letzten Proof of Concept die Möglichkeit ergänzt, eine Bildreferenz in Form eines Screenshots hochzuladen. Dieser Screenshot wird zusätzlich zum textuellen Payload an das Sprachmodell übermittelt und dient als visuelle Referenz für Layout, Farbgebung und Proportionen. Es werden ausschließlich PNG- und JPEG-Dateien akzeptiert. Zudem ist die Dateigröße auf maximal 3 MB begrenzt.

Vor der Übermittlung erfolgt eine clientseitige Vorverarbeitung des Bildes. Der Screenshot wird proportional auf maximal 768 x 768 Pixel skaliert und anschließend als komprimiertes JPEG mit einem Qualitätsfaktor von 0,75 gespeichert. Diese Maßnahme

reduziert die Datenmenge der Anfrage, ohne die für die visuelle Rekonstruktion wesentlichen Details vollständig zu verlieren. Zusätzlich wird im Interface eine Vorschau des hochgeladenen Screenshots angezeigt, die es ermöglicht, die Bildreferenz bei Bedarf wieder zu entfernen. Analog zum Zusatzhinweis wird auch die gespeicherte Bildreferenz beim erneuten Aktivieren des Selektionsmodus zurückgesetzt, um inkonsistente Zustände zu vermeiden.

Auf Serverseite wird aufgrund der zusätzlichen Bilddaten das zulässige JSON-Body-Limit auf 10 MB erhöht. Darüber hinaus wird der Payload vor der Weiterleitung an das Sprachmodell gezielt gekürzt, um die Größe der Eingabedaten zu kontrollieren und die Verarbeitung zu stabilisieren.

Trotz der Integration einer visuellen Referenz kann eine vollständige Übereinstimmung zwischen Originalelement und generierter Rekonstruktion nicht gewährleistet werden. Der zugrunde liegende Layoutkontext sowie dynamische Logikbestandteile stehen dem Modell weiterhin nur eingeschränkt zur Verfügung. Zudem erhöht die Einbindung von Bilddaten die technische Komplexität der Verarbeitungsstrecke, da zusätzliche Schritte der Validierung, Komprimierung und multimodalen Übergabe erforderlich sind.

5.2 Vergleich und Auswahl des Sprachmodells

Das Large Language Model stellt eine zentrale Komponente des entwickelten Prototyps dar, da es maßgeblich für die Generierung und Rekonstruktion der ausgewählten UI-Elemente verantwortlich ist. Die Auswahl des eingesetzten Sprachmodells erfordert daher eine fundierte und nachvollziehbare Begründung. Aus diesem Grund werden im Folgenden drei aktuelle Large Language Models hinsichtlich ihrer Eignung für das Anwendungsszenario verglichen.

Beim ersten betrachteten Modell handelt es sich um GPT-5.2 von OpenAI. Die Wissensgrenze dieses Modells liegt im August 2025. Es ist multimodal ausgelegt, wodurch sowohl textuelle als auch visuelle Eingaben verarbeitet werden können. Für die vorliegende Anwendung ist insbesondere die von GPT-5.2 unterstützte Möglichkeit strukturierter Ausgaben mittels JSON-Schema relevant, da die Browsererweiterung auf ein klar definiertes Ausgabeformat angewiesen ist. Darüber hinaus ist das Modell für Code-Generierungsaufgaben geeignet und zeigt eine stabile Verarbeitung komplexer, strukturierter Eingabedaten. Die kombinierten Kosten für eine Million Input- und Output-Token belaufen sich auf 15,75 US-Dollar und liegen damit im mittleren Bereich der betrachteten Modelle (OpenAI, 2026).

Als zweites Modell wird Sonnet 4.5 von Claude betrachtet. Auch hierbei handelt es sich um ein multimodales Sprachmodell, das sowohl Text- als auch Bilddaten verarbeiten

kann. Es eignet sich für Programmieraufgaben sowie für strukturierte Problemlösungen. Die Wissensgrenze liegt im Januar 2025. Mit Gesamtkosten von 18 US-Dollar pro eine Million Input- und Output-Token stellt es das kostenintensivste Modell im Vergleich dar (Claude, 2026).

Das letzte Modell ist Gemini 3 Pro von Google. Ebenso multimodal konzipiert, unterstützt es textuelle und visuelle Eingaben und ist für komplexe Denk- und Programmieraufgaben ausgelegt. Im Kostenvergleich weist es mit 14 US-Dollar pro einer Million Input- und Output-Token die niedrigsten Gesamtkosten der betrachteten Modelle auf (Google, 2026).

Im Folgenden wird auf Grundlage der zuvor dargestellten Modellcharakteristika ein vergleichender Test durchgeführt, um eine fundierte Auswahl des im Prototyp eingesetzten Sprachmodells zu ermöglichen. Ziel des Tests ist es, die LLMs hinsichtlich ihrer Fähigkeit zur möglichst originalgetreuen Rekonstruktion eines bestehenden UI-Elements zu bewerten. Der Schwerpunkt liegt dabei auf der visuellen Annäherung an das Original.

Alle Modelle durchlaufen denselben Testaufbau. Ihnen werden identische Eingabedaten übergeben, bestehend aus einem Prompt, einem zusätzlichen Verhaltenshinweis, einem Screenshot des Originalelements (vgl. Abbildung 5.1) sowie einem strukturierten JSON-Payload. Der Payload wird durch die Browsererweiterung aus dem selektierten UI-Element extrahiert und enthält unter anderem das originale HTML, berechnete Styles sowie Kontextinformationen.

Der verwendete Prompt lautet: “Erstelle eine Webkomponente in Vanilla JavaScript, die auf einem gegebenen HTML-Input-Element basiert. Die Komponente soll visuell möglichst nah am Original bleiben.”

Der ergänzende Zusatzhinweis lautet: “Beim hovern über das Element wird das Element leicht hervorgehoben und ein weißer Schatten entsteht.”

Durch diese standardisierte Vorgehensweise wird sichergestellt, dass Unterschiede in den Ergebnissen auf das jeweilige Modell zurückzuführen sind und nicht auf variierende Eingaben.



Abbildung 5.1: Das originale UI-Element

Die von GPT-5.2 generierte Rekonstruktion weist die größte visuelle Nähe zum Original auf (vgl. Abbildung 5.2). Die grundlegende Struktur, die Positionierung der Elemente sowie der Gesamteindruck entsprechen weitgehend der Vorlage. Abweichungen zeigen sich insbesondere in der Typografie und in der Ausgestaltung einzelner Designelemente. Die Schrift erscheint in der generierten Variante stärker gewichtet als im Original. Zudem sind die Ecken des Containers stärker abgerundet. Das “TAP TO COPY”-Element ist im Vergleich zur Vorlage etwas größer und nicht leicht geneigt, sondern gerade ausgerichtet. Der Hover-Effekt wird jedoch im Wesentlichen korrekt umgesetzt, da eine leichte Hervorhebung sowie ein weißer Schatten dargestellt werden.



Abbildung 5.2: Das rekonstruierte UI-Element mittels GPT-5.2

Die von Sonnet 4.2 generierte Variante enthält zwar alle wesentlichen Bestandteile des UI-Elements, weist jedoch deutliche Abweichungen in der Anordnung auf (vgl. Abbildung 5.3). Insbesondere ist das “TAP TO COPY”-Element nicht über dem Text positioniert, sondern seitlich daneben angeordnet. Darüber hinaus sind die Ecken des Containers ebenfalls stärker gerundet als im Original. Insgesamt wirkt die Komposition weniger präzise rekonstruiert und stärker neu interpretiert, obwohl die grundlegenden Komponenten vorhanden sind.



Abbildung 5.3: Das rekonstruierte UI-Element mittels Sonnet 4.5

Die Rekonstruktion durch Gemini 3 Pro zeigt ebenfalls erkennbare Abweichungen

vom Original (vgl. Abbildung 5.4). Besonders auffällig ist die Gestaltung des Mail-Icons, das deutliche geometrische Inkonsistenzen aufweist. Die Linienführung erscheint unpräzise und teilweise überlagernd, wodurch die Grundform eines Umschlags nur eingeschränkt erkennbar ist. Zusätzlich ist das “TAP TO COPY”-Element vertikal zu weit unten positioniert. Abgesehen von diesen Aspekten entspricht die Gesamtstruktur des Elements jedoch weitgehend der Vorlage. Insbesondere der Hover-Effekt ähnelt dem Original deutlich.



Abbildung 5.4: Das rekonstruierte UI-Element mittels Gemini 3 Pro

Zusammenfassend zeigt der Vergleich, dass alle drei Modelle grundsätzlich in der Lage sind, das UI-Element strukturell und visuell nachzubilden. Hinsichtlich der visuellen Präzision bestehen jedoch deutliche Unterschiede. GPT-5.2 erzielt im vorliegenden Testszenario die größte Übereinstimmung mit dem Original, während Sonnet 4.5 und Gemini 3 Pro stärkere Abweichungen in der Layoutgestaltung aufweisen. Auf Basis dieser Ergebnisse wird für die Browsererweiterung GPT-5.2 gewählt, da es im direkten Vergleich die präziseste visuelle Annäherung sowie die konsistenteste Einhaltung der vorgegebenen Struktur zeigt.

5.3 Konzeptionelle Entscheidungen

Die Entwicklung der Browsererweiterung basiert auf mehreren grundlegenden konzeptionellen Entscheidungen, die den erreichbaren Entwicklungsstand maßgeblich beeinflussen.

Zunächst wurde als Systemformat bewusst eine Browsererweiterung gewählt. Diese Architektur ermöglicht den direkten Zugriff auf DOM-Strukturen sowie berechnete Stilinformationen im Kontext beliebiger Webseiten. Gleichzeitig entsteht hierdurch eine strukturelle Abhängigkeit von Browser-APIs sowie deren Sicherheitsrestriktionen. Die Entscheidung erhöht somit die technische Machbarkeit der Extraktion, begrenzt jedoch

zugleich den Zugriff auf geschlossene Shadow-DOM-Strukturen, Cross-Origin-Inhalte sowie serverseitige Logik durch browserseitige Sicherheitsmechanismen.

Ein weiterer zentraler Aspekt betrifft die strukturierte Datenextraktion. Anstelle einer vollständigen Abbildung der gesamten DOM-Struktur erfolgt eine gezielte Reduktion auf rekonstruktionsrelevante Informationen. Diese Entscheidung dient der Kontrolle der Eingabekomplexität sowie der Einhaltung von Kontext- und Token-Limits des Sprachmodells. Die Payload fungiert dabei als kontrollierter Kontext und ermöglicht eine strukturierte Steuerung der Rekonstruktion. Gleichzeitig bleibt die Erfassung dynamischer Logik oder externer Abhängigkeiten prinzipiell eingeschränkt.

Eine wesentliche konzeptionelle Rolle nimmt das Prompt Engineering ein. Der finale Prompt ist in klar voneinander abgegrenzte Bestandteile strukturiert und umfasst eine vordefinierte Aufgabenbeschreibung (vgl. Anhang .2), optionale nutzerseitige Zusatzhinweise, die JSON-basierte Payload als reine Datengrundlage sowie systemseitige Rollen- und Sicherheitsanweisungen.

Dabei werden unterschiedliche Vertrauensstufen definiert. Die vordefinierten Aufgabenbeschreibungen gelten als vertrauenswürdige Instruktionen, da sie systemseitig kontrolliert erstellt und nicht durch Nutzende verändert werden können. Demgegenüber werden die frei formulierbaren Zusatzhinweise der Nutzenden aus sicherheitstechnischer Perspektive als nicht vertrauenswürdig eingestuft und entsprechend behandelt. Sie werden zwar in die Anfrage integriert, jedoch durch die Systemanweisungen klar von der eigentlichen Aufgabeninstruktion abgegrenzt.

Die JSON-Payload wird ebenfalls explizit als Dateneingabe ohne Instruktionscharakter gekennzeichnet. Dadurch wird eine semantische Trennung zwischen Steuerungsebene und Datenebene etabliert. Ziel dieser Struktur ist es, das Risiko manipulativer Eingaben oder sogenannter Prompt-Injection-Angriffe zu reduzieren, indem ausschließlich die vertrauenswürdigen Komponenten als verbindliche Steuerungsanweisungen gelten.

Systemseitige Anweisungen definieren zusätzlich die Rolle des Modells als “Frontend Code Generator” und legen Sicherheitsregeln fest, die unter anderem die Offenlegung systeminterner Informationen untersagen sowie Anweisungen zur Veränderung der Modellrolle ignorieren. Auf diese Weise wird die Generierung sowohl strukturell als auch sicherheitstechnisch kontrolliert.

Es stehen zwei aufgabenspezifische Anweisungsvarianten zur Verfügung, die entweder die Generierung einer eigenständigen Webkomponente (vgl. Anhang .2.1) oder die Erstellung eines minimalen HTML/CSS-Snippets (vgl. Anhang .2.2) vorsehen. Beide Varianten verfolgen das Ziel einer visuell möglichst nahen Rekonstruktion des Originalelements unter explizitem Ausschluss nicht rekonstruierbarer Business-Logik.

Zur strukturellen Kontrolle der Modellantwort wird ein strikt definiertes JSON-Schema eingesetzt. Das Modell ist angewiesen, ausschließlich einen Code-String ohne zusätzliche Erklärtexte oder Formatierungen zurückzugeben. Diese formale Einschränkung gewährleistet eine stabile maschinelle Weiterverarbeitung der Modellantwort innerhalb der Anwendung.

Die Integration eines Screenshots stellt eine konzeptionelle Erweiterung des Eingabekontexts dar. Der Screenshot wird als visuelle Referenz ohne optische Zeichenerkennung definiert. Dadurch entsteht ein multimodaler Kontext aus Text- und Bildinformationen. Das Modell wird explizit angewiesen, die Bildinformation nicht als Instruktion, sondern ausschließlich als visuelle Orientierung zu interpretieren.

Insgesamt zeigt sich, dass die Strukturierung des Eingabekontexts sowie das Prompt Engineering maßgeblich bestimmen, welche Rekonstruktionsqualität erreichbar ist. Trotz präziser Steuerung verbleibt ein unvermeidbarer Interpretationsspielraum des Modells, wodurch visuelle Abweichungen nicht vollständig ausgeschlossen werden können.

5.4 Analyse der Rekonstruktionsfähigkeiten

Im Rahmen der Analyse der Rekonstruktionsfähigkeit werden drei UI-Elemente unterschiedlicher Komplexität betrachtet, die im Google-Chrome-Browser extrahiert werden. Alle Testelemente stammen von der Webseite des Online-Bezahldienstes Stripe (vgl. Anhang .3) und repräsentieren eine moderne, gestaltungsorientierte Benutzeroberfläche. Die Extraktion erfolgt in einer Desktop-Ansicht mithilfe der prototypischen Browsererweiterung. Als Aufgabenstellung wird jeweils die Generierung einer Webkomponente gewählt. Auf einen Zusatzhinweis wird bewusst verzichtet, da dieser das Ergebnis maßgeblich beeinflussen kann. Ergänzend wird dem Sprachmodell ein Screenshot des jeweiligen Elements übergeben.

Beim ersten Testelement handelt es sich um einen visuell hervorgehobenen Link mit Text und Pfeilsymbol ohne komplexe strukturelle oder funktionale Logik (vgl. Abbildung 5.5). Die einzige Interaktion besteht darin, dass im Ausgangszustand lediglich die Pfeilspitze sichtbar ist, während beim Hover-Zustand der vollständige Pfeil, inklusive Stiel, angezeigt wird.



Abbildung 5.5: Leichtes Testelement zur Analyse der Rekonstruktionsfähigkeit

Die rekonstruierte Variante (vgl. Abbildung 5.6) weist eine hohe visuelle Annäherung an das Original auf. Farbgebung, Typografie sowie Abstände entsprechen weitgehend dem Ausgangselement. Auffällig ist jedoch die abweichende Darstellung des Pfeilsymbols. In der Rekonstruktion liegt die Pfeilspitze unmittelbar auf dem Stiel, während sie im Original visuell am Ende desselben positioniert ist. Zudem verändert sich der Pfeil im Hover-Zustand nicht in gleicher Weise wie im Original. Gerade bei kleinen SVG-Icons können bereits minimale Abweichungen in Rendering oder Positionierung zu wahrnehmbaren visuellen Differenzen führen. Die einzige Interaktionslogik, die konsistent übernommen wurde, ist die horizontale Verschiebung des Icons nach rechts im Hover-Zustand.

Insgesamt übernimmt der generierte Code die zentralen Inhalte der Payload, insbesondere Text, Verlinkung, relevante Attribute und grundlegende Stilmerkmale, weitgehend korrekt. Die visuelle Annäherung ist hoch, jedoch nicht vollständig originalgetreu. Technisch zeigt sich zudem eine strukturelle Abweichung. Anstelle eines einfachen nativen HTML-Ankerelements wird ein Custom Element mit Shadow DOM implementiert. Dies lässt sich dadurch erklären, dass die Aufgabenstellung häufig als Aufforderung interpretiert wird, eine wiederverwendbare und gekapselte Komponente zu erzeugen. Darüber hinaus ergänzt das Sprachmodell eine zusätzliche Interaktionslogik, indem das Element beim Anklicken eine leichte vertikale Verschiebung aufweist, die im ursprünglichen UI-Element nicht vorgesehen ist.



Abbildung 5.6: Rekonstruiertes leichtes Testelement zur Analyse der Rekonstruktionsfähigkeit

Beim mittelschweren Testelement (vgl. Abbildung 5.7) handelt es sich um ein Login-Formular, das aus mehreren UI-Komponenten besteht. Es umfasst zwei Eingabefelder

(E-Mail-Adresse und Passwort), eine Checkbox, unterschiedliche Button-Typen sowie zusätzliche Links. Das Layout ist aufgrund seiner vertikalen Struktur mit klarer visueller Hierarchie komplexer als beim leichten Testelement. Darüber hinaus weist das Element mehrere dezente Interaktionszustände auf. So verändern sich Links im Hover-Zustand in ihrer Farbgebung, der Link “Konto erstellen” wird zusätzlich unterstrichen, das Label der Checkbox passt seine Schriftfarbe an, und die alternativen Anmeldeoptionen erhalten beim Überfahren mit dem Cursor eine veränderte Umrandung. Zudem ist der Primärbutton “Anmelden” erst aktivierbar, wenn sowohl das E-Mail- als auch das Passwortfeld ausgefüllt sind.

The image shows a login form with the title "Bei Ihrem Konto anmelden". It contains the following elements from top to bottom: an "E-Mail-Adresse" input field, a "Passwort" input field with a "Passwort vergessen?" link to its right, a checked checkbox labeled "Auf diesem Gerät angemeldet bleiben", a large blue "Anmelden" button, a horizontal separator with the word "ODER" in the center, three buttons for alternative login methods: "Mit Google anmelden" (with a Google logo), "Mit Passkey anmelden", and "Mit SSO anmelden", and a footer section with the text "Neu bei Stripe? [Konto erstellen](#)".

Abbildung 5.7: Mittelschweres Testelement zur Analyse der Rekonstruktionsfähigkeit

Das rekonstruierte Element (vgl. Abbildung 5.8) zeigt visuell mehrere Abweichungen. Insbesondere sind zahlreiche Schriftgewichte im Vergleich zum Original stärker ausgeprägt, wodurch die gesamte Typografie massiver wirkt. Darüber hinaus erscheint die Karte kompakter, da die Innenabstände zwischen Inhalt und Kartenrand geringer ausfallen. Der untere, grau hinterlegte Bereich weist zudem keinen ausreichenden Abstand zum äußeren Kartenrand auf. Auch das Google-Icon ist geringfügig größer dargestellt als im originalen UI-Element.

Hinsichtlich der Interaktionslogik ergeben sich weitere Unterschiede. Die beiden Links werden in der Rekonstruktion lediglich unterstrichen, ohne ihre Schriftfarbe im Hover-

Zustand zu verändern. Beim Checkbox-Label bleibt eine Farbänderung aus, stattdessen verändert sich die Farbintensität der Checkbox selbst. Ebenso entspricht der Hover-Zustand der alternativen Anmeldeoption nicht dem Original. Während dort eine dezente Umrandungsänderung erfolgt, werden die Buttons in der Rekonstruktion flächig grau hinterlegt. Die Deaktivierung des Anmeldebuttons bei nicht ausgefüllten Eingabefeldern wurde hingegen funktional korrekt übernommen und entspricht dem Verhalten des extrahierten UI-Elements.

Wie bereits beim vorherigen Testelement wird auch hier ein Custom Element mit Shadow DOM implementiert, anstelle des im Original vorhandenen verschachtelten DOM-Baums. Zudem werden mehrere Stilwerte nicht unmittelbar aus der Payload übernommen, sondern innerhalb der Webkomponente eigenständig definiert, was zu den beschriebenen visuellen und interaktiven Abweichungen führt.

Trotz der beschriebenen Abweichungen ist das rekonstruierte UI-Element dem Original auf den ersten Blick sehr ähnlich und bildet dessen grundlegende Struktur sowie zentrale Gestaltungselemente überzeugend ab.

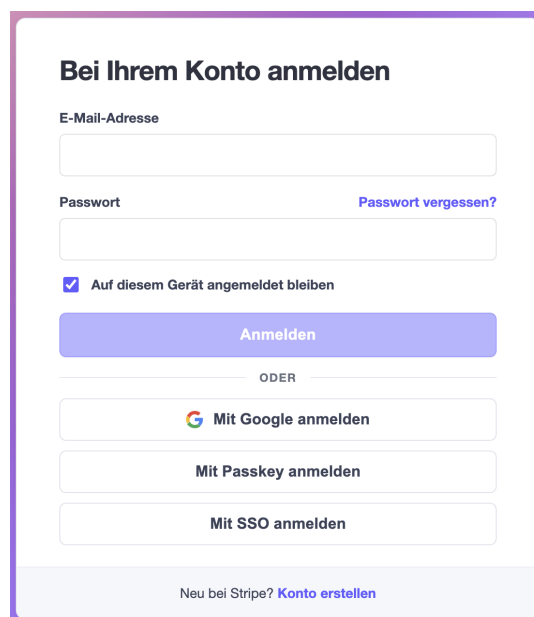
The image shows a login form titled "Bei Ihrem Konto anmelden". It contains an "E-Mail-Adresse" input field, a "Passwort" input field with a "Passwort vergessen?" link, a checked checkbox for "Auf diesem Gerät angemeldet bleiben", and a blue "Anmelden" button. Below this is a separator line with "ODER" in the center, followed by three buttons: "Mit Google anmelden" (with a Google logo), "Mit Passkey anmelden", and "Mit SSO anmelden". At the bottom, a link "Neu bei Stripe? Konto erstellen" is displayed.

Abbildung 5.8: Rekonstruiertes mittelschweres Testelement zur Analyse der Rekonstruktionsfähigkeit

Während das mittelschwere Element primär aus standardisierten UI-Komponenten besteht, kombiniert das schwere UI-Element (vgl. Abbildung 5.9) zusätzlich eine

datenbasierte Visualisierung in Form eines Balkendiagramms, eine Fortschrittsanzeige, eine mehrschichtige Kartenstruktur sowie einen mehrfarbigen Hintergrundverlauf. Die visuelle Komplexität wird durch mehrere ineinandergreifende Ebenen und eine klar ausgeprägte Hierarchie erhöht.

Im Hover-Zustand weist das Originelement mehrere gekoppelte Interaktionen auf. Die Karte vergrößert sich leicht, Diagramm und Fortschrittsanzeige reagieren mit visuellen Anpassungen, und das oben rechts platzierte Icon verändert seine Farbgebung sowie geringfügig seine Geometrie. Diese Mehrfachreaktionen erhöhen die funktionale und visuelle Komplexität des UI-Elements.

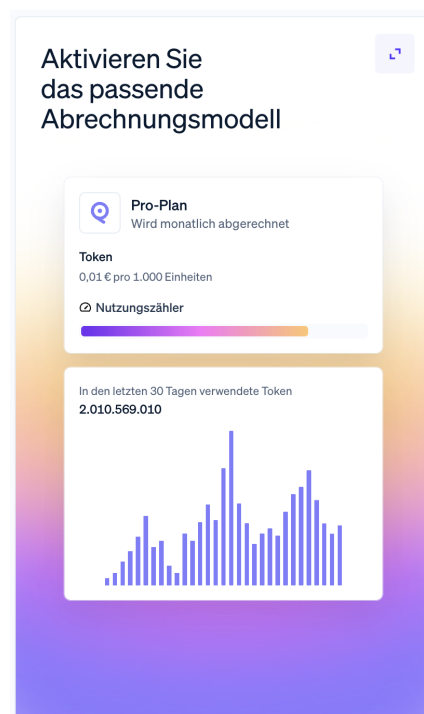


Abbildung 5.9: Schweres Testelement zur Analyse der Rekonstruktionsfähigkeit

Die rekonstruierte Variante (vgl. Abbildung 5.10) erscheint auf den ersten Blick visuell ähnlich. Dennoch lassen sich mehrere Abweichungen feststellen. Insbesondere weichen die Schriftgewichte vom Original ab und sind, wie bereits beim mittelschweren Element, insbesondere stärker ausgeprägt. Zudem stimmen die Proportionen einzelner Bestandteile nicht vollständig überein. Insbesondere wirken die unterhalb der Überschrift angeordneten Module breiter und höher als im Ausgangselement. Der

Hintergrundverlauf wird grundsätzlich korrekt nachgebildet, jedoch mit gedämpfteren Farbtönen und geringerer Kontrastwirkung.

Auch hinsichtlich der Interaktionslogik ergeben sich Unterschiede. So wird im rekonstruierten Element ein zusätzlicher visueller Effekt in Form eines lilafarbenen Punkts um den Cursor angezeigt, der im Original nicht vorhanden ist und als Halluzination des Sprachmodells einzuordnen ist. Zudem fehlt die spezifische Reaktion des Icons im rechten oberen Bereich. Stattdessen wird beim Hover eine leichte vertikale Verschiebung der gesamten Karte sowie eine verstärkte Schattenbildung innerhalb der inneren Inhaltselemente implementiert, was vom ursprünglichen Verhalten abweicht.

Strukturell wird, wie bei den vorherigen Testelementen, ein Custom Element mit Shadow DOM aufgebaut, anstatt der im Original vorhandenen verschachtelten DOM-Struktur. Darüber hinaus werden mehrere in der Payload enthaltene CSS-Variablen nicht übernommen, während zahlreiche visuelle Stilwerte eigenständig definiert werden. Dies führt zu den beschriebenen visuellen und interaktiven Differenzen.

Zusammenfassend gelingt es der Rekonstruktion, das Grundprinzip, die inhaltliche Struktur und den allgemeinen visuellen Eindruck des schweren UI-Elements nachvollziehbar abzubilden. Die komplexe Interaktionslogik und die feingliedrigen visuellen Details werden jedoch nur teilweise getroffen, sodass trotz hoher Annäherung keine vollständige, originalgetreue Reproduktion vorliegt.

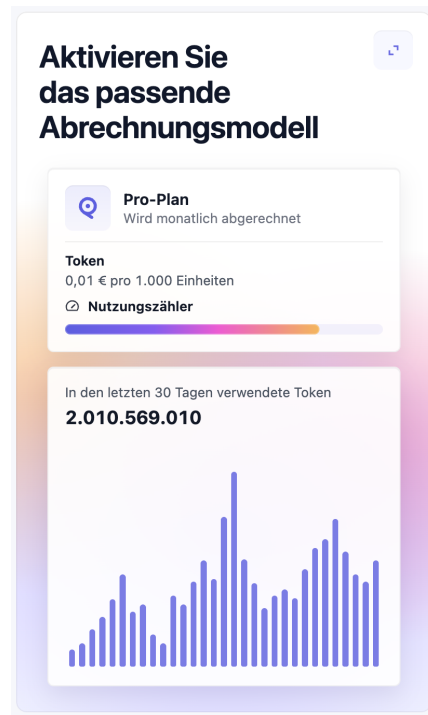


Abbildung 5.10: Rekonstruiertes schweres Testelement zur Analyse der Rekonstruktionsfähigkeit

Der durchgeführte Test zeigt, dass die Rekonstruktion bestehender UI-Elemente mithilfe eines Large Language Models grundsätzlich möglich ist. Die generierten Elemente weisen eine hohe visuelle Annäherung auf und übernehmen zentrale strukturelle sowie inhaltliche Merkmale, zeigen jedoch Abweichungen im Detail sowie in der Interaktionslogik. Die Ergebnisse legen nahe, dass die Rekonstruktionsqualität unter anderem von der Komplexität des jeweiligen UI-Elements und der Struktur der extrahierten Informationen beeinflusst wird.

6 Fazit und Ausblick

In dem folgenden Kapitel werden die zentralen Ergebnisse der vorliegenden Arbeit im Hinblick auf die zugrunde liegende Forschungsfrage eingeordnet und kritisch reflektiert. Dabei erfolgt eine zusammenfassende Bewertung der gewonnenen Erkenntnisse sowie deren Bedeutung im Kontext der entwickelten prototypischen Lösung. Darüber hinaus werden die Grenzen der Arbeit dargelegt sowie mögliche Erweiterungspotentiale aufgezeigt.

6.1 Einordnung der Ergebnisse

Ziel der vorliegenden Arbeit war die Erfassung der technischen und konzeptionellen Möglichkeiten sowie Grenzen einer prototypischen, LLM-basierten Browsererweiterung zur Extraktion und Rekonstruktion von UI-Elementen. Die durchgeführten Iterationen und Tests zeigen, dass eine visuell nahe Nachbildung bestehender Weboberflächen grundsätzlich realisierbar ist. In allen analysierten Fällen konnten zentrale strukturelle, semantische und gestalterische Merkmale der Originalelemente übernommen werden.

Gleichzeitig wird deutlich, dass die Rekonstruktionsqualität mit zunehmender Komplexität der UI-Elemente variiert. Während einfache Komponenten mit geringer Interaktionslogik weitgehend präzise reproduziert werden, treten bei komplexeren Elementen verstärkt Abweichungen im Detailgrad und in Proportionen auf. Insbesondere fein abgestimmte Interaktionen und gekoppelte Hover-Effekte lassen sich nur eingeschränkt rekonstruieren.

Die Ergebnisse verdeutlichen darüber hinaus die zentrale Rolle der strukturierten Payload sowie des Prompt Engineerings. Die gezielte Auswahl rekonstruktionsrelevanter DOM- und Stilinformationen ermöglicht eine kontrollierte Kontextbereitstellung für das Sprachmodell. Dennoch verbleibt ein Interpretationsspielraum, der zu stilistischen Neuinterpretationen oder zusätzlichen Effekten führen kann. Insgesamt kann festgehalten werden, dass die entwickelte Browsererweiterung eine tragfähige Grundlage für eine LLM-gestützte UI-Rekonstruktion darstellt.

6.2 Grenzen der Arbeit und Reflexion

Trotz der erzielten Ergebnisse unterliegt die vorliegende Arbeit mehreren methodischen und technischen Einschränkungen. Die Analyse basiert auf einer begrenzten Anzahl ausgewählter UI-Elemente einer einzelnen Webseite und erlaubt damit keine allgemeingültige Aussage über die Rekonstruktionsfähigkeit in unterschiedlichen Systemkontexten oder Designarchitekturen. Zudem erfolgt die Bewertung primär qualitativ und visuell.

Technisch ist die entwickelte Lösung durch die Restriktionen des Browser-Ökosystems begrenzt. Der Zugriff beschränkt sich auf clientseitig verfügbare DOM- und Stilinformationen, während zugrunde liegende Geschäftslogik oder komplexe Ereignisstrukturen nicht vollständig erfasst werden können. Darüber hinaus bleibt die Generierung modellabhängig und unterliegt einem unvermeidbaren Interpretationsspielraum.

Die Ergebnisse der Arbeit bieten zugleich Anknüpfungspunkte für eine weiterführende Ausgestaltung des Konzepts. Eine Weiterentwicklung der Anwendung könnte die Generierung frameworkspezifischer Komponenten ermöglichen, beispielsweise für React, Vue oder Angular. Dadurch würde die praktische Anschlussfähigkeit der generierten Ergebnisse im professionellen Entwicklungsumfeld deutlich erhöht.

Darüber hinaus wäre ein umfassenderer Testaufbau denkbar. Hierzu zählen strukturierte Nutzertests zur Bewertung der wahrgenommenen visuellen Ähnlichkeit sowie sicherheitstechnische Prüfungen hinsichtlich möglicher Prompt-Injection-Szenarien. Ebenso könnte der Einfluss einzelner Eingabekomponenten, etwa von Screenshots oder nutzerseitigen Zusatzhinweisen, systematisch untersucht werden, um deren Beitrag zur Rekonstruktionsqualität differenziert zu bewerten.

Insgesamt verdeutlicht die Arbeit sowohl das Potenzial als auch die gegenwärtigen Grenzen einer LLM-basierten UI-Rekonstruktion. Die entwickelte Lösung stellt einen funktionsfähigen Prototyp dar, dessen Weiterentwicklung insbesondere in einer vertieften Evaluation, einer Erweiterung des funktionalen Umfangs sowie in einer weiteren Absicherung der technischen Rahmenbedingungen ansetzen kann.

Literatur

- Barenkamp, M. (2023). Einflussnahme großer Sprachmodelle auf die moderne Arbeitswelt. *Informatik Spektrum*, 46(4), 185–188. <https://doi.org/10.1007/s00287-023-01546-8>
- Claude. (2026). *Sonnet 4.5* [Modell-Dokumentation]. Verfügbar 8. Januar 2026 unter [https://www.claude.com/platform/api\\$0](https://www.claude.com/platform/api$0)
- Dudenredaktion. (o.J.). *Sprachmodell*. Verfügbar 14. Februar 2026 unter [https://www.duden.de/rechtschreibung/Sprachmodell\\$0](https://www.duden.de/rechtschreibung/Sprachmodell$0)
- Epping, T. (2011). *Kanban für die Softwareentwicklung*. Springer Berlin, Heidelberg. <https://doi.org/10.1007/978-3-642-22595-6>
- Frisbie, M. (2022). *Building Browser Extensions*. Apress Berkeley, CA. <https://doi.org/10.1007/978-1-4842-8725-5>
- Google. (2026). *Gemini 3 Pro* [Modell-Dokumentation]. Verfügbar 8. Januar 2026 unter [https://ai.google.dev/gemini-api/docs/pricing?hl=de\\$0](https://ai.google.dev/gemini-api/docs/pricing?hl=de$0)
- Gupta, R., Tiwari, S., & Chaudhary, P. (2025). *Generative AI: Techniques, Models and Applications*. Springer Cham. <https://doi.org/10.1007/978-3-031-82062-5>
- Hanser, E. (2010). *Agile Prozesse: Von XP über Scrum bis MAP*. Springer Berlin, Heidelberg. <https://doi.org/10.1007/978-3-642-12313-9>
- Leible, S., Gücük, G.-L., Simic, D., von Brackel-Schmidt, C., & Lewandowski, T. (2024). Zwischen Forschung und Praxis: Fähigkeiten und Limitationen generativer KI sowie ihre wachsende Bedeutung in der Zukunft. *HMD Praxis der Wirtschaftsinformatik*, 61(2), 344–370. <https://doi.org/10.1365/s40702-024-01050-x>
- Mehta, P. (2016). *Creating Google Chrome Extensions*. Apress Berkeley, CA. <https://doi.org/10.1007/978-1-4842-1775-7>
- OpenAI. (2026). *GPT-5.2* [Modell-Dokumentation]. Verfügbar 8. Januar 2026 unter <https://platform.openai.com/docs/models/gpt-5.2>
- Schnell, P. (2024). *ChatGPT in Softwareprojekten*. O'Reilly Verlag. [https://content-select.com/de/portal/media/view/66cc3b99-27bc-412d-a949-4d55ac1b0006?forceauth=1\\$0](https://content-select.com/de/portal/media/view/66cc3b99-27bc-412d-a949-4d55ac1b0006?forceauth=1$0)
- Smutny, P., & Bojko, M. (2024). Comparative Analysis of Chatbots Using Large Language Models for Web Development Tasks. *Applied Sciences*, 14(21), Artikel 10048. <https://doi.org/10.3390/app142110048>
- Teubner, T., Flath, C. M., Weinhardt, C., van der Aalst, W., & Hinz, O. (2023). Welcome to the Era of ChatGPT et al. *Business & Information Systems Engineering*, 65(2), 95–101. <https://doi.org/10.1007/s12599-023-00795-x>

Zhao, W. X. (2025). Introduction. In *Large Language Models* (S. 3–20). Springer Nature Singapore. https://doi.org/10.1007/978-981-96-6259-3_1

Anhang

.1 Repository

Das folgende Repository enthält den vollständigen Quellcode sowie ergänzende Artefakte der im Rahmen dieser Arbeit entwickelten Browsererweiterung. <https://github.com/cosimazink/bachelorarbeit>

.2 Verwendete Prompt-Vorlagen

.2.1 Webkomponente

TASK: Create a single-file Web Component using Vanilla JavaScript.

GOAL: Reconstruct the selected UI element as visually close as possible to the original.

CONTEXT: You do NOT have access to the original JavaScript logic. You must infer visual structure and interaction behavior solely from the provided data:

- outerHTML
- computed styles
- hover and focus style snapshots
- attributes and basic text content

INSTRUCTIONS:

- Build a fully self-contained component (no external libraries, no frameworks).
- Reconstruct layout, styling, and simple interactions (e.g. hover/focus effects) by inference.
- Do NOT attempt to replicate business logic or network behavior.

REQUIRED OUTPUT:

- One complete file (e.g. index.html or component.js)
- Must include:
- HTML template
- Scoped CSS
- JavaScript

.2.2 Code-Snippet

TASK: Create a minimal HTML + CSS snippet that visually replicates the selected element.

GOAL: Achieve maximum visual similarity with minimal code.

CONTEXT: No JavaScript logic is available. Reconstruct the element using only:

- outerHTML
- computed styles
- hover/focus style snapshots

INSTRUCTIONS:

- Focus on structure, spacing, typography, and visual states.
- Keep the output as compact as possible.
- Do NOT include JavaScript.

OUTPUT RULES:

- Return ONLY raw HTML and CSS.

.3 Webseite der Testelemente

Der nachfolgende Link führt zur Webseite, auf der sich die originalen UI-Elemente befinden, die für die Analyse der Rekonstruktionsfähigkeit herangezogen wurden.
<https://stripe.com/de>

Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer oder der Verfasserin/des Verfassers selbst entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Anmerkung: In einigen Studiengängen findet sich die Erklärung unmittelbar hinter dem Deckblatt der Arbeit.

Gummersbach, 26.02.26
Ort, Datum

Gil
Unterschrift